JOBSHEET 08

CLIENT SIDE RENDERING & DATA FETCHING

PEMROGRAMAN BERBASIS FRAMEWORK

REIKA AMALIA SYAHPUTRI

2341720173

TI-3E / 19

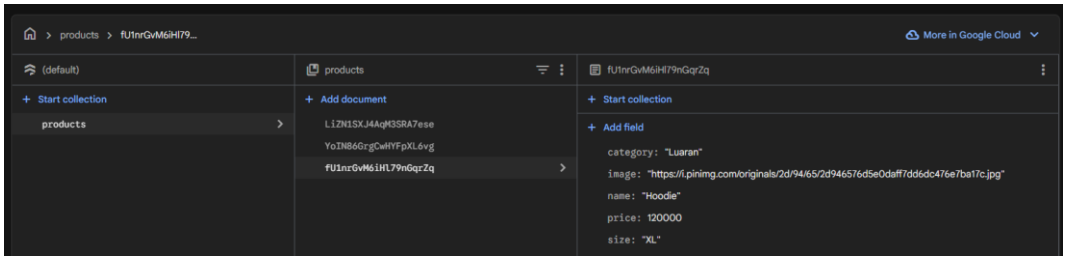
TEKNOLOGI INFORMASI

Github:

https://github.com/ReikaAmalia/Pemrograman_Berbasis_Framework/tree/main/praktikum_08

C. Langkah Praktikum

Buka firebase dan rubah fieldnya serta tambahkan beberapa field seperti pada gambar dan untuk image bisa gunakan url pada image toko-toko sepatu (copy image address). Selain itu tambah juga documentnya sehingga ada 2 document

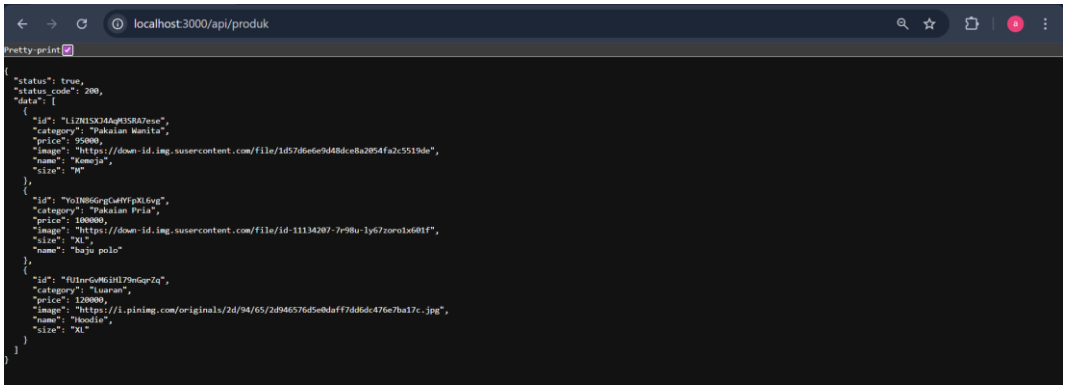


Bagian 1 – Setup Data Produk

1. Siapkan project Next.js.
2. Buat endpoint API /api/products.
3. Pastikan data memiliki:

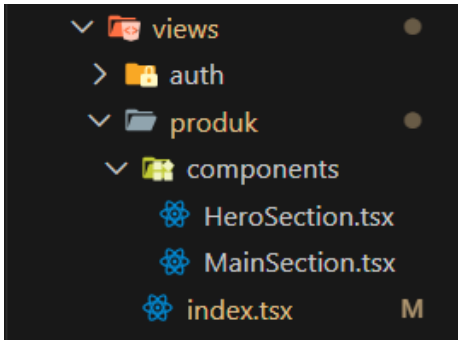
- id
- name
- category
- price
- image

4. jalankan browser http://localhost:3000/api/produk



Bagian 2 – Implementasi CSR dengan useEffect

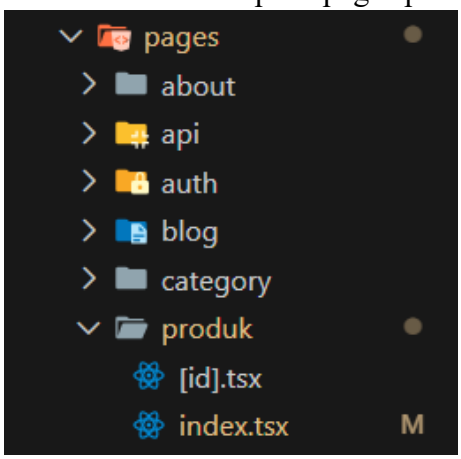
1. Membuat file index.tsx pada folder views/products



2. Modifikasi index.tsx

```
1
2  type ProductType = {
3    id: string;
4    name: string;
5    price: number;
6    image: string;
7    category: string;
8  };
9
10 const TampilanProduk = ({ products }: { products: ProductType[] }) => {
11   return (
12     <div>
13       <h1>Daftar Produk</h1>
14       {products.map((products: ProductType) => (
15         <div key={products.id}>
16           <h2>nama : {products.name}</h2>
17           <p>Harga: {products.price}</p>
18           <img src={products.image} alt={products.name} width={200} />
19           <p>kategori: {products.category}</p>
20         </div>
21       ))}
22     </div>
23   );
24 };
25
26 export default TampilanProduk;
```

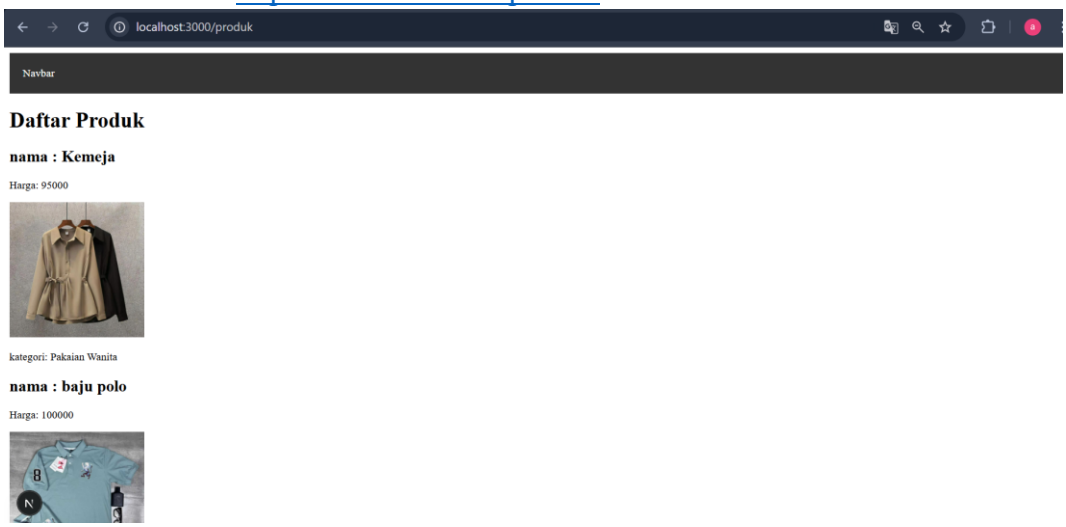
3. Buka file index.tsx pada pages/produk/



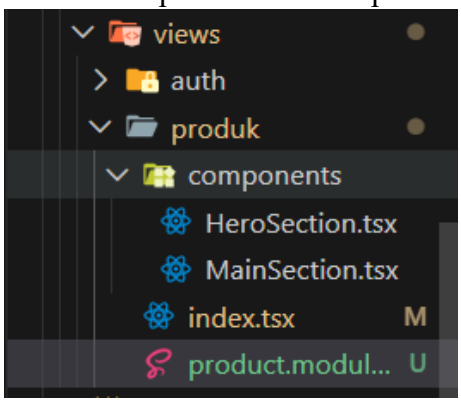
4. Modifikasi index.tsx pada pages/produk/

```
import { useRouter } from "next/router";
import { useEffect, useState } from "react";
import TampilanProduk from "@/views/produk";


const kategori = () => {
    const [products, setProducts] = useState([]);

    useEffect  (() => {
    fetch("/api/produk")
      .then((response) => response.json())
      .then((responsdata) => {
        setProducts(responsdata.data);
      })
      .catch((error) => {
        console.error("Error fetching produk:", error);
      });
  }, []);


    return (
        <div>
        <TampilanProduk products={products} />

        </div>
    );
};

export default kategori;
```

5. Jalankan browser http://localhost:3000/produk



6. Pada folder produk buat file produk.modules.scss



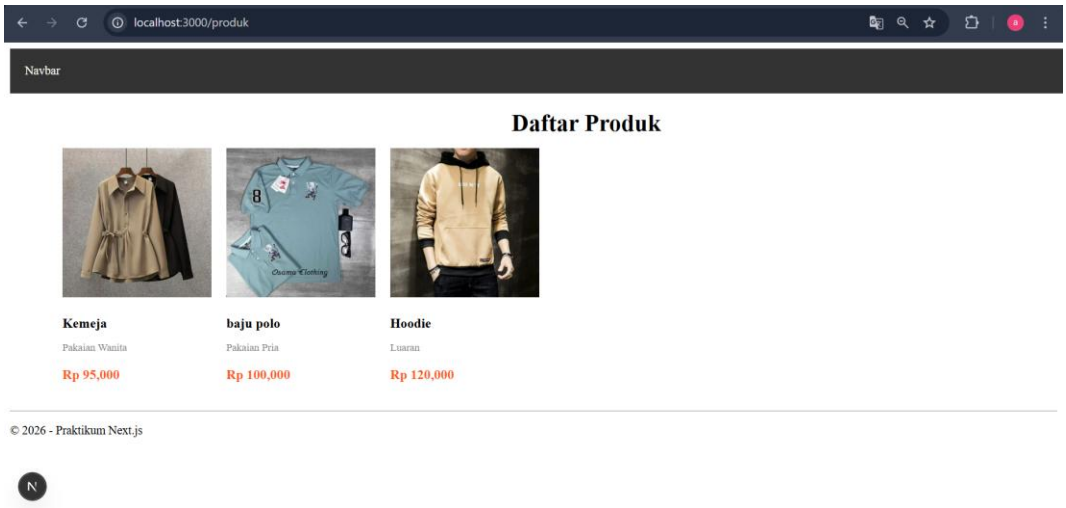
7. Modifikasi produk.modules.scss

```scss
.produk{
    width: 100%;
    padding: 0 5%;

    &__title {
    text-align: center;
    font-size: 32px;
    font-weight: bold;
    margin-bottom: 16px;
    }

    &__content {
    display: flex;
    flex-wrap: wrap;
    gap: 20px;

    &__item {
        &__image {
        width: 100%;
        height: 200px;
        object-fit: cover;
        border-radius: 4px;
        margin-bottom: 12px;
        }

        &__name {
        font-size: 18px;
        font-weight: 600;
        margin-bottom: 12px;
        }

        &__price {
        font-size: 18px;
        color: #ff5722;
        font-weight: bold;
        }

        &__category {
        font-size: 14px;
        color: #888;
        margin-bottom: 8px;
        }
    }
    }
}
```

8. Modifikasi Pada file index.tsx pada folder pages/views/product

```typescript
import styles from "../../views/produk/product.module.scss";

type ProductType = {
  id: string;
  name: string;
  price: number;
  image: string;
  size: string;
  category: string;
};

const TampilanProduk = ({ products }: { products: ProductType[] }) => {
  return (
    <div className={styles.produk}>
      <h1 className={styles.produk__title}>Daftar Produk</h1>
      <div className={styles.produk__content}>
        {products.map((products: ProductType) => (
          <div key={products.id} className={styles.produk__content__item}>
            <div className={styles.produk__content__item__image}>
              <img src={products.image} alt={products.name} width={200} />
            </div>
            <h4 className={styles.produk__content__item__name}>
              {products.name}
            </h4>
            <p className={styles.produk__content__item__category}>
              {products.category}
            </p>
            <p className={styles.produk__content__item__price}>
              Rp {products.price.toLocaleString()}
            </p>
          </div>
        ))}
      </div>
    </div>
  );
};

export default TampilanProduk;
```

9. Jalankan Browser



Bagian 3 – Implementasi Skeleton Loading

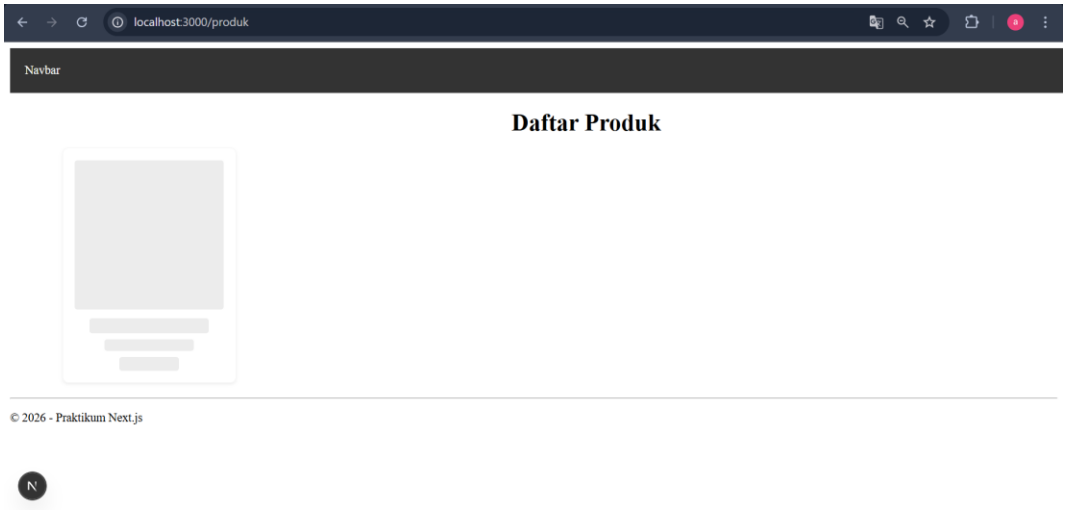
- Modfikasi file index.tsx pada folder views/product/index.tsx

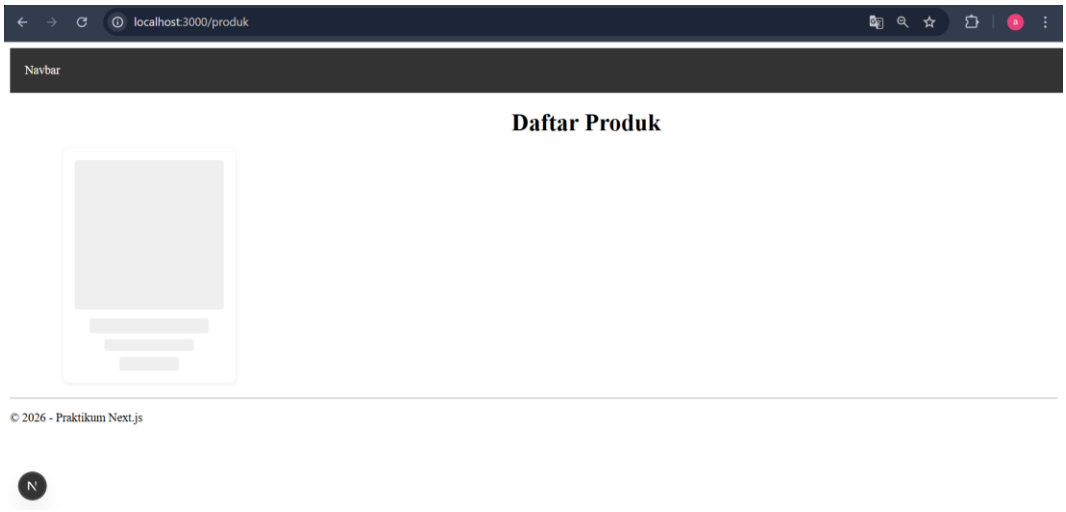
```tsx
import styles from "../../views/produk/product.module.scss";

type ProductType = {
  id: string;
  name: string;
  price: number;
  image: string;
  size: string;
  category: string;
};

const TampilanProduk = ({ products }: { products: ProductType[] }) => {
  return (
    <div className={styles.produk}>
      <h1 className={styles.produk__title}>Daftar Produk</h1>
      <div className={styles.produk__content}>
        {/* {products.map((products: ProductType) => (
          <div key={products.id} className={styles.produk__content__item}>
            <div className={styles.produk__content__item__image}>
              <img src={products.image} alt={products.name} width={200} />
            </div>
            <h4 className={styles.produk__content__item__name}>
              {products.name}
            </h4>
            <p className={styles.produk__content__item__category}>
              {products.category}
            </p>
            <p className={styles.produk__content__item__price}>
              Rp {products.price.toLocaleString()}
            </p>
          </div>
        ))} */}
        <div className= {styles.produk_content_skeleton}>
          <div className={styles.produk_content_skeleton_image}></div>
          <div className={styles.produk_content_skeleton_name}></div>
          <div className={styles.produk_content_skeleton_category}></div>
          <div className={styles.produk_content_skeleton_price}></div>
        </div>
      </div>
    </div>
  );
};

export default TampilanProduk;
```

- Modifikasi file product.module.scss

```css
.produk{
    width: 100%;
    padding: 0 5%;

    &__title {
    text-align: center;
    font-size: 32px;
    font-weight: bold;
    margin-bottom: 16px;
    }

    &__content {
    display: flex;
    flex-wrap: wrap;
    gap: 20px;

     &__skeleton {
      width: 200px;
      padding: 16px;
      border: 1px solid #eee;
      border-radius: 8px;
      box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
      display: flex;
      flex-direction: column;
      align-items: center;
      animation: identifier 1.5s infinite ease-in-out;

      &__image {
        width: 100%;
        height: 200px;
        background-color: #e0e0e0;
        border-radius: 4px;
        margin-bottom: 12px;
      }

      &__name {
        width: 80%;
        height: 20px;
        background-color: #e0e0e0;
        border-radius: 4px;
        margin-bottom: 8px;
      }

      &__category {
        width: 60%;
        height: 16px;
        background-color: #e0e0e0;
        border-radius: 4px;
        margin-bottom: 8px;
      }

      &__price {
        width: 40%;
        height: 18px;
        background-color: #e0e0e0;
        border-radius: 4px;
      }
    }

    &__item {
        &__image {
        width: 100%;
        height: 200px;
        object-fit: cover;
        border-radius: 4px;
        margin-bottom: 12px;
        }

        &__name {
        font-size: 18px;
        font-weight: 600;
        margin-bottom: 12px;
        }

        &__price {
        font-size: 18px;
        color: #ff5722;
        font-weight: bold;
        }

        &__category {
        font-size: 14px;
        color: #888;
        margin-bottom: 8px;
        }
    }
    }
}

@keyframes identifier {
    0% {
        opacity: 1;
    }
    50% {
        opacity: 0.5;
    }
    100% {
        opacity: 1;
    }
}
```

- Jalankan browser maka akan muncul skeleton yang terdapat animasi berkedip

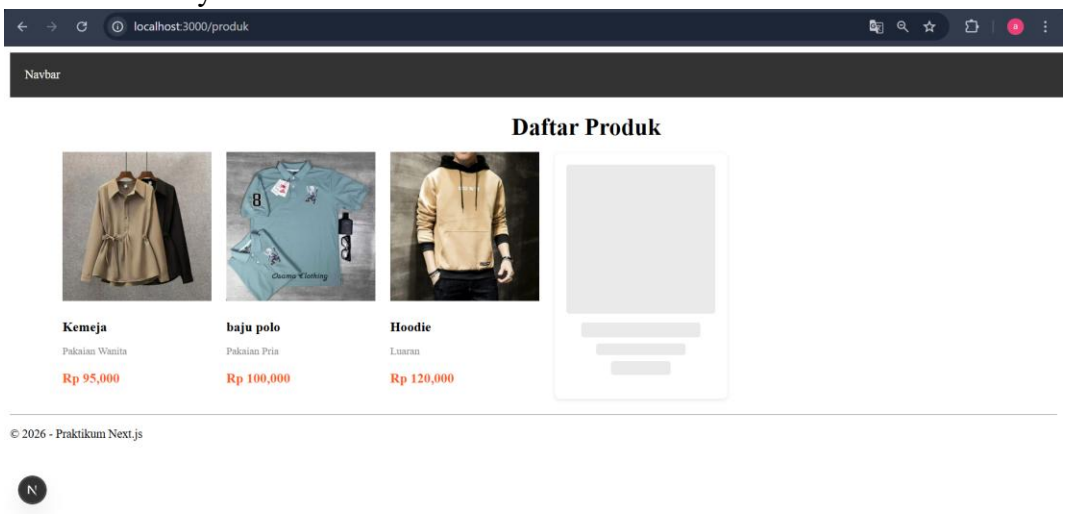




- Modifikasi pada index.tsx pada folder views/product/index.tsx

```
import styles from "../../styles/product.module.scss";

type ProductType = {
  id: string;
  name: string;
  price: number;
  image: string;
  category: string;
};

const TampilanProduk = ({ products }: { products: ProductType[] }) => {
  return (
    <div className={styles.produk}>
      <h1 className={styles.produk__title}>Daftar Produk</h1>
      <div className={styles.produk__content}>
        {products.map((products: ProductType) => (
          <div key={products.id} className={styles.produk__content__item}>
            <div className={styles.produk__content__item__image}>
              <img src={products.image} alt={products.name} width={200} />
            </div>
            <h4 className={styles.produk__content__item__name}>
              {products.name}
            </h4>
            <p className={styles.produk__content__item__category}>
              {products.category}
            </p>
            <p className={styles.produk__content__item__price}>
              Rp {products.price.toLocaleString()}
            </p>
          </div>
        ))}
        <div className={styles.produk__content__skeleton}>
          <div className={styles.produk__content__skeleton__image}></div>
          <div className={styles.produk__content__skeleton__name}></div>
          <div className={styles.produk__content__skeleton__category}></div>
          <div className={styles.produk__content__skeleton__price}></div>
        </div>
      </div>
    </div>
  );
};

export default TampilanProduk;
```

- Jalankan browser
  Jika dijalankan akan muncul skeletonnya terlebih dahulu setelah itu muncul gambar dan informasinya



Bagian 5 – Implementasi SWR

https://swr.vercel.app/

1. Install SWR

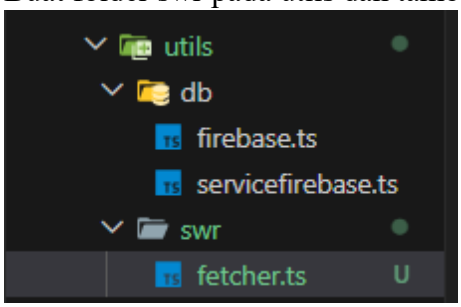
```
PS D:\Pemograman_Framework\praktikum_08\my-app-08> npm install swr
npm WARN EBADENGINE Unsupported engine {
npm WARN EBADENGINE   package: 'eslint-visitor-keys@5.0.1',
npm WARN EBADENGINE   required: { node: '^20.19.0 || ^22.13.0 || >=24' },
npm WARN EBADENGINE   current: { node: 'v20.11.1', npm: '10.2.4' }
npm WARN EBADENGINE }

added 3 packages, and audited 432 packages in 2s

144 packages are looking for funding
  run `npm fund` for details

1 high severity vulnerability

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
```

2. Buka dan modifkasi file index.tsx pada folder pages/product/

```
import { useRouter } from "next/router";
import { useEffect, useState } from "react";
import TampilanProduk from "@/views/produk";
import useSWR from "swr";

const fetcher = (url: string) => fetch(url).then((res) => res.json());
const kategori = () => {
    const [products, setProducts] = useState([]);
    const { data, error, isLoading } = useSWR("/api/produk", fetcher);

    return (
        <div>
        <TampilanProduk products={isLoading ? [] : data.data } />
        </div>
    );
};

export default kategori;
```

3. Agar terlihat lebih rapi

- Buat folder swr pada utils dan tambahkan file dengan nama fetcher.js



- Modifikasi file fetcher.ts

```
const fetcher = (url: string) => fetch(url).then((res) => res.json());

export default fetcher;
```

- Modifikasi file index.tsx pada folder pages/produk

```
import { useRouter } from "next/router";
import { useEffect, useState } from "react";
import TampilanProduk from "@/views/produk";
import useSWR from "swr";
import fetcher from "../utils/swr/fetcher";

// const fetcher = (url: string) => fetch(url).then((res) => res.json());
const kategori = () => {
    const [products, setProducts] = useState([]);
    const { data, error, isLoading } = useSWR("/api/produk", fetcher);


    return (
        <div>
        <TampilanProduk products={isLoading ? [] : data.data } />
        </div>
    );
};

export default kategori;
```

Bandingkan:

• useEffect manual

• SWR

Jawab:

Pengambilan data menggunakan useEffect manual dilakukan dengan memanggil `fetch()` di dalam hook lalu mengatur sendiri state untuk data, loading, dan error, sehingga kodenya lebih panjang dan semua proses dikontrol secara manual. Sedangkan menggunakan SWR lebih sederhana karena cukup memanggil `useSWR()` dan library tersebut sudah menyediakan fitur loading, error handling, caching, dan revalidation secara otomatis, sehingga kode lebih ringkas dan rapi. Jadi, useEffect cocok untuk memahami dasar cara kerja CSR, sementara SWR lebih praktis dan efisien untuk pengembangan aplikasi yang lebih kompleks.

Note : untuk client side rendering dapat menggunakan 2 cara yaitu menggunakan manual (userEffect) atau menggunakan SWR

Tugas Individu

1. Jelaskan perbedaan:

- Client Side Rendering
- Server Side Rendering
- Static Site Generation

Jawab:

Client Side Rendering (CSR) adalah proses rendering halaman yang dilakukan di sisi browser. Pada metode ini, server hanya mengirimkan file HTML dasar dan JavaScript, lalu data diambil setelah halaman dimuat melalui proses fetch di client. Biasanya ada sedikit delay sebelum data tampil karena proses pengambilan data dilakukan setelah halaman terbuka. Server Side Rendering (SSR) berbeda karena proses rendering dilakukan di server terlebih dahulu. Jadi, saat pengguna membuka halaman, server langsung mengirimkan HTML yang sudah berisi data lengkap, sehingga konten bisa langsung terlihat tanpa menunggu fetch di browser. Sedangkan Static Site Generation (SSG) adalah metode di mana halaman sudah dibuat dan di-generate saat proses build, bukan saat ada request dari user. Artinya, halaman sudah dalam bentuk HTML statis dan sangat cepat diakses, tetapi datanya tidak berubah secara langsung kecuali dilakukan build ulang.

2. Buat halaman produk dengan:

- Skeleton loading
- Animasi

Jawab:

- Data diambil menggunakan SWR
- Skeleton ditampilkan menggunakan conditional rendering berdasarkan isLoading
- Animasi dibuat menggunakan @keyframes pada CSS
- Setelah data tersedia, skeleton digantikan oleh data produk

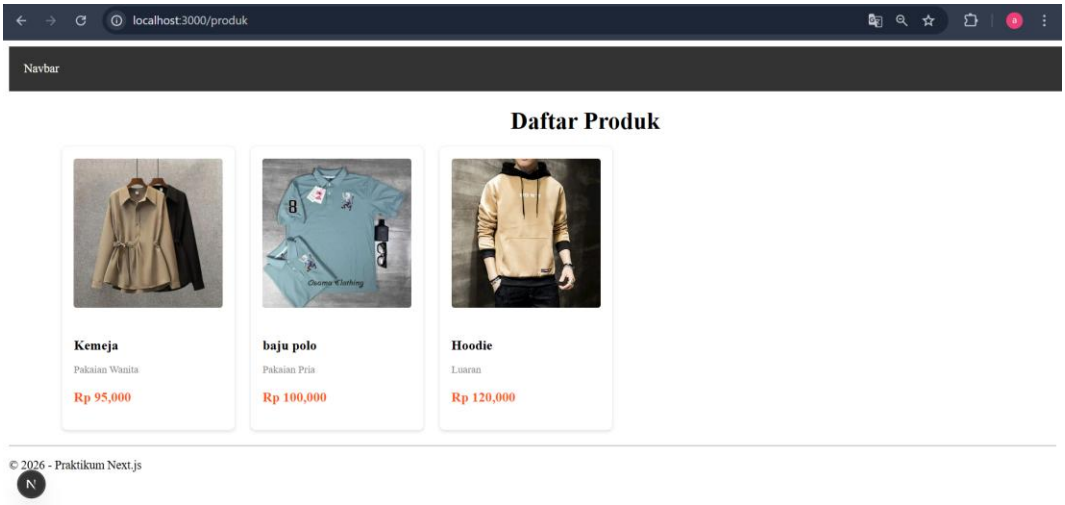
Modifikasi pada views/produk/index.tsx

```
import styles from "../../styles/product.module.scss";

type ProductType = {
  id: string;
  name: string;
  price: number;
  image: string;
  category: string;
};

const TampilanProduk = ({ products, isLoading, }: { products: ProductType[]; isLoading?: boolean }) => {
  return (
    <div className={styles.produk}>
      <h1 className={styles.produk__title}>Daftar Produk</h1>

      <div className={styles.produk__content}>

        {isLoading ? (
          <>
            <div className={styles.produk__content__skeleton}>
              <div className={styles.produk__content__skeleton__image}></div>
              <div className={styles.produk__content__skeleton__name}></div>
              <div className={styles.produk__content__skeleton__category}></div>
              <div className={styles.produk__content__skeleton__price}></div>
            </div>

            <div className={styles.produk__content__skeleton}>
              <div className={styles.produk__content__skeleton__image}></div>
              <div className={styles.produk__content__skeleton__name}></div>
              <div className={styles.produk__content__skeleton__category}></div>
              <div className={styles.produk__content__skeleton__price}></div>
            </div>
          </>
        ) : (
          products.map((product: ProductType) => (
            <div
              key={product.id}
              className={styles.produk__content__item}
            >
              <div className={styles.produk__content__item__image}>
                <img
                  src={product.image}
                  alt={product.name}
                  width={200}
                />
              </div>

              <h4 className={styles.produk__content__item__name}>
                {product.name}
              </h4>

              <p className={styles.produk__content__item__category}>
                {product.category}
              </p>

              <p className={styles.produk__content__item__price}>
                Rp {product.price.toLocaleString()}
              </p>
            </div>
          ))
        )}

      </div>
    </div>
  );
};

export default TampilanProduk;
```

Modifikasi pada product.module.scss

```scss
.produk {
    width: 100%;
    padding: 0 5%;

    &__title {
        text-align: center;
        font-size: 32px;
        font-weight: bold;
        margin-bottom: 16px;
    }

    &__content {
        display: flex;
        flex-wrap: wrap;
        gap: 20px;

        &__item {
        width: 200px;
        padding: 16px;
        border: 1px solid #eee;
        border-radius: 8px;
        box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);

        &__image img {
            width: 100%;
            height: 200px;
            object-fit: cover;
            border-radius: 4px;
            margin-bottom: 12px;
        }

        &__name {
            font-size: 18px;
            font-weight: 600;
            margin-bottom: 8px;
        }

        &__category {
            font-size: 14px;
            color: #888;
            margin-bottom: 8px;
        }

        &__price {
            font-size: 18px;
            color: #ff5722;
            font-weight: bold;
        }
        }

        &__skeleton {
        width: 200px;
        padding: 16px;
        border: 1px solid #eee;
        border-radius: 8px;
        display: flex;
        flex-direction: column;
        align-items: center;
        animation: blink 1.5s infinite ease-in-out;

        &__image {
            width: 100%;
            height: 200px;
            background-color: #e0e0e0;
            border-radius: 4px;
            margin-bottom: 12px;
        }

        &__name {
            width: 80%;
            height: 20px;
            background-color: #e0e0e0;
            border-radius: 4px;
            margin-bottom: 8px;
        }

        &__category {
            width: 60%;
            height: 16px;
            background-color: #e0e0e0;
            border-radius: 4px;
            margin-bottom: 8px;
        }

        &__price {
            width: 40%;
            height: 18px;
            background-color: #e0e0e0;
            border-radius: 4px;
        }
        }
    }
    }

    @keyframes blink {
    0% {
        opacity: 1;
    }
    50% {
        opacity: 0.5;
    }
    100% {
        opacity: 1;
    }
    }
```

Output:



3. Refactor kode dari useEffect menjadi SWR.

```
const { data, error, isLoading } = useSWR("/api/produk", fetcher);
```